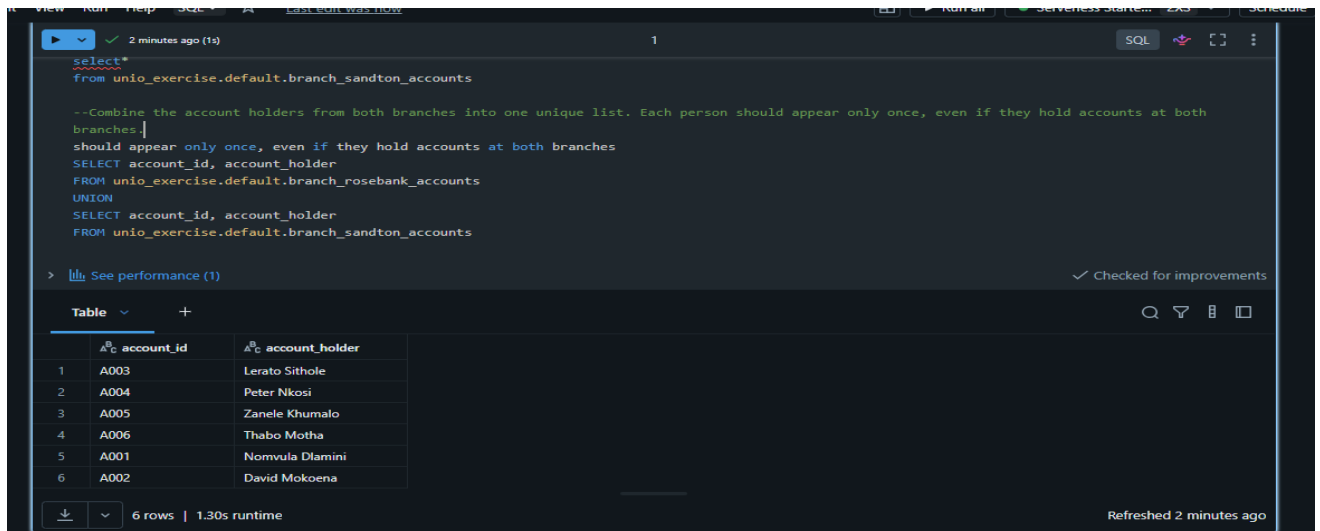


1. Combine the account holders from both branches into one unique list. Each person should appear only once, even if they hold accounts at both branches.



The screenshot shows a SQL IDE with a query that combines account holders from two branches into a unique list. The query uses a UNION to merge data from 'branch_sandton_accounts' and 'branch_rosebank_accounts'. The result table shows 6 rows of account holders, with no duplicates.

```

select*
from unio_exercise.default.branch_sandton_accounts

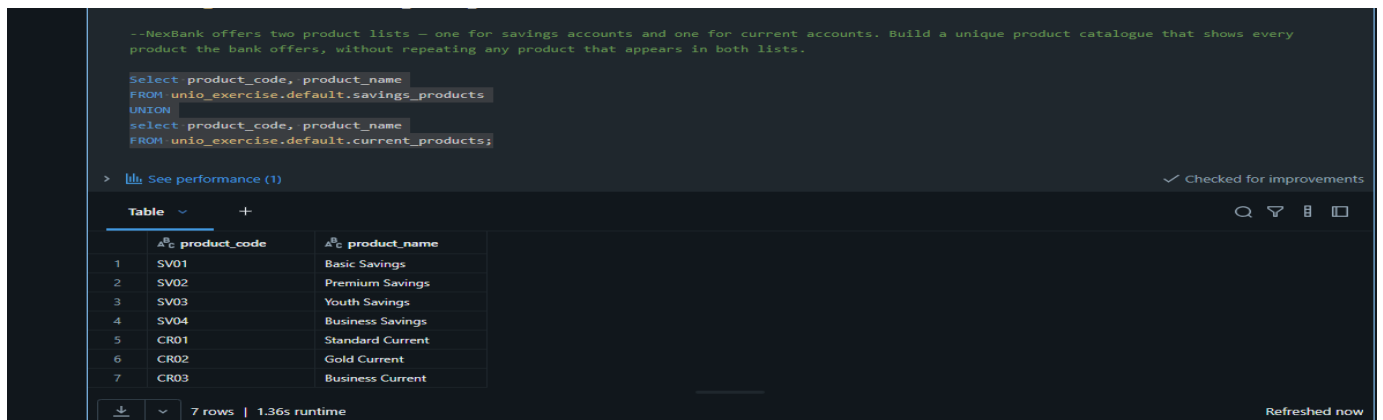
--Combine the account holders from both branches into one unique list. Each person should appear only once, even if they hold accounts at both
branches.
should appear only once, even if they hold accounts at both branches
SELECT account_id, account_holder
FROM unio_exercise.default.branch_rosebank_accounts
UNION
SELECT account_id, account_holder
FROM unio_exercise.default.branch_sandton_accounts

```

	account_id	account_holder
1	A003	Lerato Sithole
2	A004	Peter Nkosi
3	A005	Zanele Khumalo
4	A006	Thabo Motha
5	A001	Nomvula Dlamini
6	A002	David Mokoena

6 rows | 1.30s runtime

2. NexBank offers two product lists — one for savings accounts and one for current accounts. Build a unique product catalogue that shows every product the bank offers, without repeating any product that appears in both lists.



The screenshot shows a SQL IDE with a query that builds a unique product catalogue by combining savings and current products. The query uses a UNION to merge data from 'savings_products' and 'current_products'. The result table shows 7 rows of products, with no duplicates.

```

--NexBank offers two product lists — one for savings accounts and one for current accounts. Build a unique product catalogue that shows every
product the bank offers, without repeating any product that appears in both lists.

Select product_code, product_name
FROM unio_exercise.default.savings_products
UNION
select product_code, product_name
FROM unio_exercise.default.current_products;

```

	product_code	product_name
1	SV01	Basic Savings
2	SV02	Premium Savings
3	SV03	Youth Savings
4	SV04	Business Savings
5	CR01	Standard Current
6	CR02	Gold Current
7	CR03	Business Current

7 rows | 1.36s runtime

3. The HR team needs a unique list of all staff who work in either Retail Banking or Corporate Banking. Some staff are assigned to both. Each staff member should appear only once.

```
--The HR team needs a unique list of all staff who work in either Retail Banking or Corporate Banking. Some staff are assigned to both. Each staff member should appear only once.
```

```
select staff_id, staff_name
FROM unio_exercise.default.retail_banking_staff
UNION
select staff_id, staff_name
FROM unio_exercise.default.corporate_banking_staff;
```

> [See performance \(1\)](#) ✓ Checked for improvements

	staff_id	staff_name
1	S001	Mpho Radebe
2	S002	Brian Tshabalala
3	S003	Aisha Patel
4	S004	Kabelo Moabelo
5	S005	Nandi Dube
6	S006	Sipho Khumalo

6 rows | 1.09s runtime Refreshed 1 minute ago

4. Marketing needs a unique list of all cities where NexBank serves customers — either through its mobile branches or its digital branch (which serves customers online regardless of city). No city should appear twice.

```
--Marketing needs a unique list of all cities where NexBank serves customers — either through its mobile branches or its digital branch (which serves customers online regardless of city). No city should appear twice.
```

```
select city_code, city_name
from unio_exercise.default.mobile_branch_cities
UNION
select city_code, city_name
from unio_exercise.default.digital_branch_cities;
```

> [See performance \(1\)](#) ✓ Checked for improvements

	city_code	city_name
1	C01	Johannesburg
2	C02	Pretoria
3	C03	Cape Town
4	C04	Durban
5	C05	Polokwane
6	C06	Port Elizabeth

5. The digital marketing team ran two campaigns last month — one via push notification and one via in-app banner. Build a unique list of every customer who was targeted. If a customer received both, they must appear only once.

```
--The digital marketing team ran two campaigns last month — one via push notification and one via in-app banner. Build a unique list of every customer who was targeted. If a customer received both, they must appear only once.
```

```
Select customer_id, customer_name
from unio_exercise.default.push_notification_targets
UNION
select customer_id, customer_name
from unio_exercise.default.inapp_banner_targets;
```

> [See performance \(1\)](#) ✓ Checked for improvements

	customer_id	customer_name
1	C1001	Nomsa Zwane
2	C1002	Andile Buthelezi
3	C1003	Fatima Mahomed
4	C1004	Ryno van Zyl
5	C1005	Thandeka Cele
6	C1006	Samuel Nkosi

6. The fraud team needs the complete log of all ATM transactions from machine ATM-01 and machine ATM-02. Every transaction must appear — even if the same transaction appears in both logs due to a system sync error.

```
--The fraud team needs the complete log of all ATM transactions from machine ATM-01 and machine ATM-02. Every transaction must appear – even if the same transaction appears in both logs due to a system sync error.
```

```

select txn_id, account_id, amount
from unio_exercise.default.atm_01_transactions
UNION ALL
select txn_id, account_id, amount
from unio_exercise.default.atm_02_transactions

```

> [See performance \(1\)](#) ✓ Checked for improvements

	txn_id	account_id	amount
1	T1001	A001	500
2	T1002	A002	1200
3	T1003	A001	300
4	T1004	A003	750
5	T1003	A001	300
6	T1005	A004	900
7	T1006	A002	450
8	T1007	A005	150

7. The credit team wants a full list of all loan applications received from the Gauteng region and the Western Cape region. Every application must be included — duplicates are allowed because both regions may have captured the same application.

```
--The credit team wants a full list of all loan applications received from the Gauteng region and the Western Cape region. Every application must be included – duplicates are allowed because both regions may have captured the same application.
```

```

select app_id, customer_id, loan_type
FROM unio_exercise.default.gauteng_loan_applications
UNION ALL
select app_id, customer_id, loan_type
from unio_exercise.default.western_cape_loan_applications

```

> [See performance \(1\)](#) ✓ Checked for improvements

	app_id	customer_id	loan_type
1	LA001	C1001	Home Loan
2	LA002	C1002	Vehicle Loan
3	LA003	C1003	Personal
4	LA004	C1004	Home Loan
5	LA003	C1003	Personal
6	LA005	C1005	Vehicle Loan
7	LA006	C1006	Business
8	LA007	C1007	Home Loan

8. The customer experience team needs a full record of every complaint logged this month, whether it came via email or via the mobile app. Every complaint must appear — the team needs all of them to calculate total complaint volumes.

```
--The customer experience team needs a full record of every complaint logged this month, whether it came via email or via the mobile app. Every complaint must appear – the team needs all of them to calculate total complaint volumes.
```

```

select complaint_id, customer_id
from unio_exercise.default.email_complaints
UNION ALL
select complaint_id, customer_id
from unio_exercise.default.app_complaints

```

> [See performance \(1\)](#) ✓ Checked for improvements

	complaint_id	customer_id
1	EC001	C2001
2	EC002	C2002
3	EC003	C2003
4	EC004	C2004
5	AC001	C2005
6	AC002	C2001
7	AC003	C2006
8	AC004	C2007

9. Finance needs to reconcile all payment records from April and May into one table for the half-year report. Every payment from both months must appear — the total row count matters for reconciliation.

```
--Finance needs to reconcile all payment records from April and May into one table for the half-year report. Every payment from both months must appear — the total row count matters for reconciliation.
```

```
select payment_id, account_id, amount
from unio_exercise.default.april_payments
UNION ALL
select payment_id, account_id, amount
from unio_exercise.default.may_payments
```

> [See performance \(1\)](#) ✓ Checked for improvements

	payment_id	account_id	amount
1	PAY001	A001	12500
2	PAY002	A002	4800
3	PAY003	A003	9200
4	PAY004	A004	3300
5	PAY005	A001	12500
6	PAY006	A005	7600
7	PAY007	A002	5100
8	PAY008	A006	2800

10. The accounting system records debits and credits in two separate tables. Combine all entries into one general ledger for the audit. Every single entry from both tables must appear — no rows should be dropped.

```
--The accounting system records debits and credits in two separate tables. Combine all entries into one general ledger for the audit. Every single entry from both tables must appear — no rows should be dropped.
```

```
select entry_id, account_id, entry_type
from unio_exercise.default.debit_entries
UNION ALL
select entry_id, account_id, entry_type
from unio_exercise.default.credit_entries
```

> [See performance \(1\)](#) ✓ Checked for improvements

	entry_id	account_id	entry_type
1	DR001	A001	Debit
2	DR002	A002	Debit
3	DR003	A003	Debit
4	DR004	A004	Debit
5	CR001	A001	Credit
6	CR002	A005	Credit
7	CR003	A002	Credit
8	CR004	A006	Credit

Bonus questions

1. A fraud analyst wants a unique list of customers who appeared in either the January or February high-risk watch lists. Should they use UNION or UNION ALL? Explain why.

Answer: The UNION function should be used, because we want to distinctly show each customer once in the list and union function allows us to remove the duplicates.

2. The auditing team wants to count every individual transaction that passed through the bank in Q1 — even if the same transaction was recorded in two systems on the same day. Should they use UNION or UNION ALL? Explain why.

Answer: The UNION ALL function should be used because it also retains duplicate entries and the audit team wants to count all individual transactions even if recorded in the two different systems.

3. A developer writes this query: `SELECT account_id, account_holder FROM branch_sandton_accounts UNION SELECT account_id, account_holder, city FROM branch_rosebank_accounts;` The query fails. What is wrong, and how would you fix it?

Answer: The second select statement has got column that does not exist in the first select statement, it means these tables do not have identical columns and UNION can only run if columns contained in both tables are similar. To fix this, the city column should be removed from the Syntex second select statement.